

**UNIVERSIDADE ESTADUAL DE PONTA GROSSA
SETOR DE ENGENHARIAS, CIÊNCIAS AGRÁRIAS E DE
TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA**

**Carlos Eduardo Varela de Almeida
Mateus Enrick Pitura
Raissa Mayara Moreira**

**Impacto do WebAssembly no desempenho de
aplicações Rust**

**PONTA GROSSA
2026**

Carlos Eduardo Varela de Almeida

Mateus Enrick Pitura

Raissa Mayara Moreira

Impacto do WebAssembly no desempenho de aplicações Rust

Trabalho de Conclusão de Curso
apresentado ao Curso de Engenharia
de Software da Universidade Estadual
de Ponta Grossa, como requisito parcial
para obtenção do título de Bacharel em
Engenharia de Software.

Orientador: Prof. Dr. Luciano José Senger

PONTA GROSSA
2026

AGRADECIMENTOS

Expressamos nossos sinceros agradecimentos ao Prof. Dr. Luciano José Senger pela orientação dedicada ao longo deste trabalho, conduzida com atenção, paciência e compromisso acadêmico. Suas contribuições foram essenciais para o desenvolvimento e a conclusão desta pesquisa.

Aos nossos pais e amigos, manifestamos nossa profunda gratidão pelo apoio incondicional, incentivo constante e confiança depositada durante toda a trajetória acadêmica. A presença de vocês foi fundamental para a concretização deste trabalho.

RESUMO

O WebAssembly tem se consolidado como uma alternativa para execução de aplicações de alto desempenho em navegadores *web*, permitindo a utilização de linguagens compiladas além da linguagem JavaScript. Entre elas, o Rust destaca-se pela eficiência computacional e sua segurança de memória, tornando-se uma opção importante para o desenvolvimento de aplicações executadas no ambiente *web*. Neste contexto, este trabalho realiza uma análise comparativa entre a execução nativa de algoritmos implementados em Rust e a execução dos mesmos algoritmos compilados para WebAssembly. Para a realização dos experimentos, foram selecionados algoritmos representativos de diferentes classes de problemas computacionais a partir do The Computer Language Benchmarks Game. As implementações foram adaptadas para os dois ambientes de execução e avaliadas em ambiente controlado por contêineres Docker, com análise de tempo de execução e consumo de memória. Os resultados mostram que o Rust nativo foi mais rápido em todos os algoritmos avaliados, com *overhead* do WebAssembly variando de 10,9% a 126,9% conforme o algoritmo, com mediana de 21,2%. Em relação à memória, o ambiente WebAssembly manteve cerca de 210 MiB em repouso, devido ao navegador e à ferramenta de automação, o que tornou necessário o uso das métricas de *baseline* e consumo líquido para isolar o consumo dos algoritmos, que foi reduzido na maioria dos casos e maior no mandelbrot, nos dois ambientes, em valor compatível com o *buffer* de saída do algoritmo. Conclui-se que o WebAssembly pode ser uma alternativa viável para aplicações Rust executadas no navegador, mas que o custo adicional depende fortemente da carga de trabalho, devendo ser avaliado para cada aplicação quando o desempenho for um requisito crítico. Os resultados contribuem para a compreensão dos impactos do WebAssembly em aplicações Rust e auxiliam na tomada de decisões sobre quando utilizar essa tecnologia em aplicações executadas no navegador.

Palavras-chave: Rust; WebAssembly; desempenho; benchmarking; execução nativa; Docker; Puppeteer.

LISTA DE FIGURAS

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos .	26
Figura 2 – Comparativo de tempo de execução médio	34

LISTA DE TABELAS

Tabela 1 – Comparativo de tempo de execução médio (ms)	33
Tabela 2 – Consumo de memória do executável Rust (MiB)	35
Tabela 3 – Consumo de memória do executável WebAssembly (MiB)	35
Tabela A1 – Tempo médio de execução e <i>overhead</i> por computador	41
Tabela A2 – Consumo de memória por computador (MiB)	41

LISTA DE ALGORITMOS

Algoritmo 1	–	Exemplo de <i>ownership</i> : cada valor tem um único dono	13
Algoritmo 2	–	Violação das regras de <i>borrowing</i> em Rust	14
Algoritmo 3	–	Referências separadas por escopo, <i>borrowing</i> correto	15
Algoritmo 4	–	Acesso fora dos limites e encerramento por panic	16
Algoritmo 5	–	Tipo <i>Option</i> no lugar de ponteiro nulo	17
Algoritmo 6	–	Função Rust exportada para WebAssembly	20
Algoritmo 7	–	Chamada da função Rust a partir do JavaScript	20
Algoritmo 8	–	Memória liberada automaticamente ao fim do escopo	21
Algoritmo 9	–	Função Rust em Wasm recebendo <i>string</i> do JavaScript	22

LISTA DE ABREVIATURAS E SIGLAS

Wasm	WebAssembly
W3C	World Wide Web Consortium
HTML	HyperText Markup Language
URL	Uniform Resource Locator
HTTPS	Hypertext Transfer Protocol Secure
JIT	Just-In-Time
LLVM	Low Level Virtual Machine
CPU	Central Processing Unit
RAM	Random Access Memory
SSD	Solid State Drive
NVMe	Non-Volatile Memory Express
SATA	Serial AT Attachment
WSL	Windows Subsystem for Linux
GB	Gigabyte
MiB	Mebibyte
MHz	Megahertz

SUMÁRIO

1	INTRODUÇÃO	10
1.1	OBJETIVOS	10
1.1.1	Objetivo geral	10
1.1.2	Objetivos Específicos	10
1.2	JUSTIFICATIVA	11
2	REVISÃO DA LITERATURA	12
2.1	RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE	12
2.1.1	Segurança: o Modelo de Ownership e Borrowing	12
2.2	WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS	18
2.2.1	Arquitetura e Funcionamento	18
2.2.2	Segurança	18
2.2.3	Compilação de Rust para WebAssembly	19
2.3	ESTUDOS COMPARATIVOS DE DESEMPENHO	21
2.3.1	Rust: Desempenho sem Runtime ou Garbage Collector	21
2.3.2	WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa	22
3	MATERIAL E MÉTODOS	24
3.1	PESQUISA BIBLIOGRÁFICA	24
3.2	IMPLEMENTAÇÃO PRÁTICA	24
3.2.1	Algoritmos	25
3.2.2	Configuração dos Testes	25
3.3	AMBIENTE DE TESTE	25
3.3.1	Hardwares	26
3.3.2	Softwares	27
3.4	BENCHMARKING	27
3.4.1	Coleta de métricas	28
3.4.2	Tempo de execução	29
3.4.3	Consumo de memória	29
3.4.4	Consolidação dos resultados	31
4	ANÁLISE DOS RESULTADOS	33
4.1	TEMPO DE EXECUÇÃO	33
4.2	USO DE MEMÓRIA	35
4.3	SÍNTESE COMPARATIVA	36
4.4	LIMITAÇÕES DO ESTUDO	36
5	CONSIDERAÇÕES FINAIS	38

REFERÊNCIAS 39

APÊNDICE A – RESULTADOS POR COMPUTADOR 40

1 INTRODUÇÃO

O crescimento das aplicações *web* modernas e a necessidade cada vez maior de desempenho no lado do cliente têm tornado a escolha do ambiente de execução um aspecto importante no desenvolvimento de software. Sendo assim, o WebAssembly tem se destacado por permitir a execução de código compilado diretamente no navegador com desempenho próximo ao nativo. Paralelo a isso, a linguagem Rust vem sendo utilizada como uma das principais opções para gerar código nesse formato.

Mesmo com sua popularidade, a execução de aplicações Rust por meio do WebAssembly ainda apresenta limitações, já que o ambiente do navegador impõe restrições que não existem na execução nativa. Além disso, ainda há poucos estudos experimentais que analisem de forma prática o impacto dessa abordagem sobre o desempenho.

Diante disso, surge a questão: qual é o impacto da execução de aplicações Rust via WebAssembly no desempenho quando comparado à execução nativa, considerando diferentes tipos de problemas computacionais, e em quais situações essa abordagem pode ser considerada viável?

O objetivo deste trabalho é comparar a execução nativa de programas escritos em Rust com a execução dos mesmos programas compilados para WebAssembly no navegador, analisando o impacto em desempenho e consumo de recursos. Para isso, foram utilizados algoritmos representativos do The Computer Language Benchmarks Game (Fulgham; Gouy, 2026), adaptados para execução nos dois ambientes e avaliados sob condições controladas para coleta de métricas comparáveis.

1.1 OBJETIVOS

1.1.1 Objetivo geral

Este trabalho tem como objetivo geral realizar uma análise comparativa entre a execução de código Rust e a execução do mesmo código compilado para WebAssembly no navegador, avaliando os impactos em desempenho e consumo de recursos.

1.1.2 Objetivos Específicos

- Selecionar algoritmos representativos de diferentes classes de problemas computacionais a partir do The Computer Language Benchmarks Game (Fulgham; Gouy, 2026), adequados para execução nos dois ambientes avaliados;

- Avaliar o consumo de recursos computacionais e tempo de execução de cada um dos algoritmos escolhidos;
- Elaborar diretrizes sobre em quais cenários a execução via WebAssembly representa uma alternativa viável à execução nativa.

1.2 JUSTIFICATIVA

O avanço das aplicações *web* modernas tem impulsionado a busca por tecnologias capazes de oferecer desempenho próximo ao nativo diretamente no navegador. Nesse cenário, a combinação entre Rust e WebAssembly desponta como uma alternativa promissora, reunindo as garantias de segurança e eficiência da linguagem à portabilidade do formato binário suportado pelos principais navegadores.

Embora existam estudos comparando execução nativa e WebAssembly, estes concentram-se majoritariamente em linguagens como C e C++, deixando Rust pouco explorado nesse contexto (Jangda *et al.*, 2019). Este trabalho se justifica, portanto, tanto pelo valor técnico de produzir dados comparativos concretos sobre os dois modelos de execução quanto pela contribuição ao campo da Engenharia de Software, oferecendo subsídios para decisões arquiteturais em projetos que avaliem a viabilidade de adotar WebAssembly como ambiente de execução.

2 REVISÃO DA LITERATURA

Nesta seção são abordados os principais fundamentos e características técnicas das tecnologias Rust e WebAssembly. Inicialmente, apresenta-se a linguagem Rust, sua performance e sua proposta de garantir segurança de memória sem o uso de coletor de lixo, delegando o controle de alocação ao compilador por meio do modelo de *ownership*. Em seguida, discute-se o uso de WebAssembly como padrão binário de baixo nível suportado pelos principais navegadores, abordando sua arquitetura, modelo de execução e o processo de compilação de código Rust para esse formato.

2.1 RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE

Rust é uma linguagem de programação de sistemas desenvolvida a partir da necessidade de conciliar alto desempenho e segurança no desenvolvimento de software. Desde que a Mozilla começou a patrocinar seu desenvolvimento em 2010, a linguagem foi ganhando espaço na comunidade devido à forma como ela lida com dois aspectos historicamente desafiadores no desenvolvimento de software: o desempenho e a segurança de memória.

2.1.1 Segurança: o Modelo de Ownership e Borrowing

Se o desempenho de Rust já se mostra expressivo, é na segurança que a linguagem realmente se destaca. Estima-se que cerca de 70% das vulnerabilidades de segurança catalogadas pelo Microsoft Security Response Center têm origem em erros de gerenciamento de memória (Bugden; Alahmar, 2022). Nesse contexto, o Rust aborda diretamente esse problema por meio de um mecanismo denominado *ownership* (propriedade).

O conceito é baseado em um princípio: cada valor em Rust tem um único dono, responsável por seu gerenciamento. Quando o proprietário deixa de existir ou sai de escopo, o valor associado é automaticamente desalocado da memória. Isso impede, por exemplo, que um programa tente usar um dado que já foi descartado, erro conhecido como *use after free*, comum em C e frequentemente explorado por hackers.

O Algoritmo 1 apresenta um exemplo do funcionamento do *ownership*. Inicialmente, a variável `nome` é proprietária de uma `String`. Quando essa variável é atribuída a linguagem, a propriedade do valor é transferida, caracterizando uma operação de *move*. A partir desse momento, `"nome"` deixa de ser uma variável válida para acessar o valor, enquanto `"linguagem"` passa a ser sua proprietária.

Algoritmo 1 – Exemplo de *ownership*: cada valor tem um único dono

```
fn main() {  
    let nome = String::from("Rust");  
    // O ownership de 'nome' é transferido para 'linguagem'.  
    let linguagem = nome;  
    // A linha abaixo causaria erro de compilação,  
    // porque 'nome' não é mais o dono do valor.  
    println!("{}", nome); // erro!  
    println!("{}", linguagem); // funciona normalmente  
}
```

Fonte: Os autores

A tentativa de utilizar "nome" após a transferência de propriedade resulta em erro de compilação, pois o valor já pertence à variável "linguagem". Dessa forma, o compilador impede que uma variável continue sendo utilizada como se ainda fosse proprietária de um recurso que já foi transferido.

Quando há necessidade de compartilhar um valor entre diferentes partes do programa sem transferir o *ownership*, existe o *borrowing* (empréstimo). Nesse mecanismo, uma parte do código pode acessar temporariamente um valor pertencente a outra parte sem assumir sua propriedade. As regras são: ou várias partes do código leem o dado ao mesmo tempo, referências imutáveis, ou uma única parte pode modificá-lo, referência mutável. As duas situações nunca coexistem (Bugden; Alahmar, 2022).

O Algoritmo 2 demonstra uma situação em que essas regras são violadas. Inicialmente, é criada uma referência imutável para valores. Em seguida, o código tenta criar uma referência mutável para o mesmo valor. Como a referência imutável ainda está ativa, o compilador identifica o conflito entre os dois tipos de empréstimo.

Algoritmo 2 – Violação das regras de *borrowing* em Rust

```
fn processar_dados() {  
    // Declara uma String que pode ser modificada.  
    let mut valores = String::from("Rust");  
  
    // Cria uma referência imutável para o valor.  
    let leitura = &valores;  
  
    // Tenta criar uma referência mutável para o mesmo valor  
    // enquanto a referência imutável ainda está ativa.  
    let alteracao = &mut valores;  
  
    // Utiliza a referência imutável.  
    println!("Conteúdo: {}", leitura);  
  
    // Tenta modificar o valor por meio da referência mutável.  
    alteracao.push_str(" é seguro");  
  
    // Exibe o valor após a alteração.  
    println!("Resultado: {}", alteracao);  
}
```

Fonte: Os autores

O código apresentado no Algoritmo 2 é rejeitado pelo compilador antes de sua execução. O erro ocorre porque `valores` não pode ser emprestado de forma mutável enquanto permanece emprestado de forma imutável. Essa restrição evita que diferentes partes do programa tenham, simultaneamente, permissões incompatíveis sobre o mesmo dado.

A solução consiste em garantir que os empréstimos não se sobreponham. Para demonstrar essa situação, o Algoritmo 3 utiliza um escopo interno para limitar a duração da referência imutável. Enquanto o bloco interno está em execução, `"leitura"` pode acessar `valores`. Ao final desse bloco, a referência deixa de estar ativa, permitindo que uma referência mutável seja criada posteriormente.

Algoritmo 3 – Referências separadas por escopo, *borrowing* correto

```
fn main() {  
    // Declara uma String que pode ser modificada.  
    let mut valores = String::from("Rust");  
  
    {  
        // Cria e utiliza uma referência imutável para o valor.  
        let leitura = &valores;  
        println!("Conteúdo: {}", leitura);  
    } // A referência imutável deixa de existir neste ponto.  
  
    // Agora é seguro criar uma referência mutável,  
    // pois não há mais uma referência imutável ativa.  
    let alteracao = &mut valores;  
    alteracao.push_str(" é seguro");  
  
    println!("Resultado: {}", alteracao);  
}
```

Fonte: Os autores

Diferentemente do Algoritmo 2, o Algoritmo 3 é compilado e executado normalmente, pois a referência imutável "leitura" deixa de existir antes da criação da referência mutável "alteracao". Nesse caso, o valor pode primeiro ser consultado e, posteriormente, modificado, sem que haja sobreposição entre os empréstimos.

Essa regra é verificada pelo compilador e contribui para impedir um tipo de erro difícil de depurar chamado condição de corrida (*data race*), que ocorre quando diferentes partes de um programa tentam acessar e modificar simultaneamente uma mesma região de memória sem mecanismos adequados de sincronização (Bugden; Alahmar, 2022).

Outro ponto importante é que Rust não permite manipular endereços de memória diretamente fora de contextos explicitamente marcados como *unsafe*. Na prática, isso significa que todo acesso a listas e *arrays* é verificado automaticamente: se o programa tentar acessar uma posição inexistente, ele encerra de forma controlada em vez de continuar operando com dados corrompidos ou expondo informações que não deveria (Bugden; Alahmar, 2022). Esse tipo de proteção teria evitado o Heartbleed, uma das maiores vulnerabilidades da história da internet, que afetou uma parcela expressiva dos maiores sites HTTPS do mundo ao permitir que dados fossem lidos além dos limites permitidos da memória.

O Algoritmo 4 exemplifica esse mecanismo por meio de um vetor contendo três elementos. Como os índices válidos são 0, 1 e 2, a tentativa de acessar a posição 5 representa um acesso fora dos limites. Nesse caso, o Rust detecta a condição e interrompe a execução por meio de um *panic*.

Algoritmo 4 – Acesso fora dos limites e encerramento por panic

```
fn main() {  
    // Cria um vetor contendo três elementos.  
    let numeros = vec![10, 20, 30];  
  
    // Os índices válidos do vetor são 0, 1 e 2.  
    // O índice 5 não existe, pois está fora dos limites do vetor.  
    // O Rust detecta o acesso inválido durante a execução  
    // e encerra o programa por meio de um panic.  
    println!("{}", numeros[5]); // erro: index out of bounds  
}
```

Fonte: Os autores

Ao executar o Algoritmo 4, a compilação é realizada normalmente, mas a execução é interrompida quando o acesso ao índice inexistente é realizado. O sistema apresenta uma mensagem indicando que o vetor possui comprimento 3, enquanto o índice solicitado é 5. Esse comportamento demonstra que o Rust realiza a verificação do limite durante a execução da operação de indexação, impedindo que o acesso inválido prossiga silenciosamente.

Por fim, Rust também elimina o conceito de ponteiro nulo, aquele valor que representa "nada" e que em C e C++ frequentemente causa travamentos silenciosos. No lugar, Rust usa o tipo *Option*, que obriga o programador a decidir explicitamente o que fazer quando um valor pode não existir (Bugden; Alahmar, 2022).

O Algoritmo 5 demonstra essa abordagem por meio de uma função responsável por buscar um nome a partir de um identificador. A função retorna *Some* quando encontra um valor correspondente e *None* quando nenhum valor é encontrado. Dessa maneira, a possibilidade de ausência do resultado é representada diretamente pelo tipo de retorno da função.

Algoritmo 5 – Tipo *Option* no lugar de ponteiro nulo

```
fn buscar_nome(id: u32) -> Option<String> {  
    // Retorna um nome quando o identificador corresponde a um usuário.  
    if id == 1 {  
        Some(String::from("Olá, Mundo"))  
    } else {  
        // Indica que nenhum valor foi encontrado.  
        None  
    }  
}  
  
fn main() {  
    // Realiza uma busca utilizando um identificador que não existe.  
    match buscar_nome(2) {  
        // Executado quando a busca retorna um valor.  
        Some(nome) => println!("Encontrado: {}", nome),  
  
        // Executado quando nenhum valor é encontrado.  
        None => println!("Nenhum usuário com esse id"),  
    }  
}
```

Fonte: Os autores

No exemplo apresentado, a chamada `buscar_nome(2)` retorna `None`, pois o identificador informado não corresponde ao valor esperado pela função. O `match` trata explicitamente essa possibilidade e exibe a mensagem correspondente. Caso a função encontrasse um usuário, retornaria `Some(nome)`, e o primeiro ramo do `match` seria executado. Dessa forma, o código não pressupõe que o valor necessariamente exista, tornando essa possibilidade explícita na estrutura do programa.

O que torna tudo isso notável é que essas verificações acontecem em tempo de compilação. Erros que em outras linguagens só aparecem quando o programa já está rodando, às vezes em produção, às vezes explorados por um hacker, em Rust são detectados antes mesmo de o programa existir (Bugden; Alahmar, 2022).

Nos casos apresentados nos Algoritmos 1, 2 e 3, por exemplo, as regras de *ownership* e *borrowing* são verificadas durante a compilação, enquanto o acesso fora dos limites demonstrado no Algoritmo 4 é detectado durante a execução por meio de um *panic*. Já o Algoritmo 5 evidencia como a possibilidade de ausência de um valor pode ser representada explicitamente pelo tipo *Option*.

2.2 WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS

Se Rust resolve o problema de escrever código seguro e eficiente, o WebAssembly resolve um problema diferente: como fazer esse código ser executado dentro de um navegador com desempenho próximo ao nativo. É justamente essa combinação que torna as duas tecnologias complementares e justifica o foco deste trabalho.

Por muito tempo, o JavaScript foi a única linguagem executada pelos navegadores. Isso funcionou para páginas simples, mas com o crescimento de aplicações mais pesadas, editores de imagem, jogos, processamento de vídeo, suas limitações ficaram evidentes. Foi para resolver esse problema que Google, Microsoft, Mozilla e Apple se uniram para criar o WebAssembly, lançado em 2017 como padrão oficial do W3C (Haas *et al.*, 2017).

WebAssembly é um formato binário compacto de baixo nível que executa com desempenho próximo ao nativo, servindo como alvo de compilação para linguagens como Rust, C e C++. Ele não foi criado para substituir o JavaScript, mas para trabalhar ao lado dele: enquanto o JavaScript cuida da interface e das interações, o Wasm assume as partes que exigem mais processamento (MDN Web Docs, 2026).

2.2.1 Arquitetura e Funcionamento

Um programa WebAssembly é organizado em módulos isolados, onde o código de um módulo não acessa dados de outro a menos que isso seja explicitamente permitido. O armazenamento de dados acontece em uma estrutura chamada memória linear, um vetor de bytes completamente separado do código executável e das estruturas internas do motor de execução. Isso garante que mesmo um programa com falha não consiga corromper o ambiente ao redor (Moric; Branstett; Petrunic, 2022).

Esse isolamento tem uma consequência prática importante: módulos de origens diferentes podem executar no mesmo processo sem interferir uns nos outros, o que viabiliza o uso do Wasm em ambientes como servidores na nuvem e dispositivos embarcados, não apenas em navegadores (Moric; Branstett; Petrunic, 2022).

2.2.2 Segurança

A segurança faz parte da especificação do WebAssembly desde o início, e não foi adicionada depois como um recurso extra. Um módulo Wasm não possui acesso a nenhum recurso do sistema, arquivos, redes ou dispositivos, a menos que o ambiente hospedeiro conceda esse acesso de forma explícita. Além disso, o fluxo de execução é validado antes de o programa executar por meio de um mecanismo

chamado integridade do fluxo de controle (*Control-Flow Integrity*), que impede que código malicioso redirecione a execução para posições não autorizadas na memória (Moric; Branstett; Petrunic, 2022).

Como os métodos do WebAssembly são tipados, o uso incorreto de tipos, como passar um valor do tipo errado para uma função, não é possível, o que reforça ainda mais a confiabilidade do ambiente de execução. Quando combinado com as garantias de segurança de memória do Rust, esse modelo cria uma camada dupla de proteção: o compilador Rust elimina os erros antes da geração do binário, e o WebAssembly garante que o binário gerado rode dentro de limites seguros (Moric; Branstett; Petrunic, 2022).

2.2.3 Compilação de Rust para WebAssembly

O processo de transformar código Rust em um módulo WebAssembly é direto e bem suportado pelo ecossistema da linguagem. O fluxo básico envolve três etapas: o código é escrito em Rust normalmente, compilado para o formato `.wasm` por meio de ferramentas como o `wasm-pack`, e então carregado e executado pelo ambiente de destino, seja um navegador, um servidor ou outro *runtime* compatível.

O `wasm-pack` é a principal ferramenta para esse processo no ecossistema Rust. Ela cuida de compilar o código, gerar os arquivos de integração com JavaScript e empacotar tudo em um formato pronto para ser publicado ou importado por uma aplicação *web*. Por baixo, o compilador Rust utiliza o *backend* LLVM para gerar o binário `.wasm`, o mesmo *backend* responsável pelas otimizações que tornam o código Rust tão eficiente em ambiente nativo (Moric; Branstett; Petrunic, 2022).

Para exemplificar esse processo, o Algoritmo 6 apresenta uma função implementada em Rust e preparada para ser exportada para WebAssembly. A função calcula o n -ésimo termo da sequência de Fibonacci e utiliza o atributo `#[wasm_bindgen]` para permitir sua integração com JavaScript.

Algoritmo 6 – Função Rust exportada para WebAssembly

```

use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn fibonacci(n: u32) -> u64 {
    match n {
        0 => 0,
        1 => 1,
        _ => {
            let (mut a, mut b) = (0u64, 1u64);
            for _ in 2..=n {
                let temp = a + b;
                a = b;
                b = temp;
            }
            b
        }
    }
}

```

Fonte: Os autores

A macro `#[wasm_bindgen]` gera os *bindings* necessários para que a função apresentada no Algoritmo 6 possa ser chamada a partir do JavaScript, conforme demonstrado no Algoritmo 7.

Algoritmo 7 – Chamada da função Rust a partir do JavaScript

```

import init, { fibonacci } from "../pkg/meu_projeto.js";

async function executar() {
    await init(); // carrega e instancia o módulo .wasm
    const resultado = fibonacci(30);
    console.log(resultado);
}

executar();

```

Fonte: Os autores

Nesse caso, o JavaScript inicializa o módulo WebAssembly e, em seguida, chama a função `fibonacci`, passando o valor 30 como parâmetro. O resultado retornado pela função Rust é então exibido no console do navegador.

Vale destacar que nem todo código Rust compila para WebAssembly sem adaptações. Funcionalidades que dependem diretamente do sistema operacional, como acesso a arquivos ou criação de *threads*, não estão disponíveis no ambiente do navegador por padrão.

2.3 ESTUDOS COMPARATIVOS DE DESEMPENHO

Compreender o impacto do WebAssembly no desempenho de aplicações Rust exige fazer duas análises: primeiro, o que torna o Rust eficiente em ambiente nativo; segundo, o que acontece com esse desempenho quando o mesmo código é compilado para WebAssembly e executado dentro do navegador. As subseções a seguir percorrem essas duas camadas, formando a base comparativa que orienta a investigação empírica deste trabalho.

2.3.1 Rust: Desempenho sem Runtime ou Garbage Collector

Quando um programa é executado, ele precisa reservar espaço na memória para guardar seus dados, e também precisa liberar esse espaço quando os dados não são mais necessários. Linguagens como Java e Python fazem isso automaticamente por meio de um mecanismo chamado coletor de lixo (*garbage collector*), que fica em segundo plano verificando o que pode ser descartado. Isso é conveniente, mas tem um custo: o coletor consome processamento, ocupa memória extra e pode pausar o programa em momentos imprevisíveis para fazer sua limpeza (Bugden; Alahmar, 2022).

Rust não tem coletor de lixo. Em vez disso, o compilador analisa o código antes de o programa ser executado e já determina exatamente quando cada dado deve ser criado e quando deve ser descartado. Quando uma variável chega ao fim do bloco em que foi criada, sua memória é liberada automaticamente, sem nenhuma instrução extra do programador e sem nenhum custo em tempo de execução.

O Algoritmo 8 ilustra esse comportamento por meio de uma variável do tipo `String`. Ao atingir o final do escopo em que foi declarada, a variável deixa de existir e a memória associada ao seu valor é liberada automaticamente.

Algoritmo 8 – Memória liberada automaticamente ao fim do escopo

```
fn main() {  
    // 'mensagem' é criada aqui e pertence a este escopo  
    let mensagem = String::from("olá, Mundo!");  
    println!("{}", mensagem);  
    // Ao chegar aqui, 'mensagem' sai de escopo  
    // e sua memória é liberada automaticamente pelo compilador.  
    // Nenhuma instrução manual é necessária.  
}
```

Fonte: Os autores

O exemplo demonstra, portanto, que a liberação da memória ocorre automaticamente ao final do escopo, sem a necessidade de uma chamada explícita

para liberar o recurso.

Esse modelo elimina tanto o trabalho manual de liberar memória, que em C frequentemente gera erros, quanto a sobrecarga do coletor de lixo. O resultado é um programa que roda com eficiência comparável à de C e C++, mas sem exigir que o programador gerencie a memória manualmente (Bugden; Alahmar, 2022).

É esse nível de eficiência nativa que serve de referência para a comparação com o WebAssembly. Quando o mesmo código Rust é compilado para o formato .wasm e executado no navegador, parte dessas garantias de desempenho precisa ser reexaminada, pois o ambiente de execução impõe restrições e custos que não existem na compilação nativa.

2.3.2 WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa

Dentro do ambiente do navegador, o WebAssembly já representa um avanço considerável em relação ao JavaScript. Em tarefas computacionais intensas, o WebAssembly supera o JavaScript, sendo em média 1,3 vezes mais rápido que o JavaScript tradicional, seu antecessor mais próximo (Jangda *et al.*, 2019). No entanto, há uma ressalva importante: transferir dados complexos para dentro da memória do módulo tem um custo. Para passar uma *string* ao Wasm, por exemplo, é necessário codificá-la, localizar espaço livre na memória linear e escrevê-la *byte a byte*, um processo que pode anular o ganho de desempenho em operações simples com texto. Quando o dado já está na memória ou o algoritmo é suficientemente complexo, o Wasm volta a superar o JavaScript por ampla margem (Moric; Branstett; Petrunic, 2022).

Para ilustrar essa interação entre JavaScript e WebAssembly, o Algoritmo 9 apresenta uma função Rust que recebe uma *string* como parâmetro e realiza a contagem de suas vogais. O exemplo evidencia o uso de dados textuais na comunicação entre o código JavaScript e o módulo WebAssembly.

Algoritmo 9 – Função Rust em Wasm recebendo *string* do JavaScript

```
use wasm_bindgen :: prelude :: * ;

#[wasm_bindgen]
pub fn contar_vogais(texto : &str) -> u32 {
    texto
        .chars()
        .filter(|c| "aeiouAEIOU".contains(*c))
        .count() as u32
}
```

Fonte: Os autores

Contudo, a comparação mais relevante para este trabalho não é entre WebAssembly e JavaScript, mas entre WebAssembly e a execução nativa do próprio Rust. Nesse confronto, o cenário é diferente. Estudos mostram que aplicações compiladas para WebAssembly são executadas, em média, entre 45% e 55% mais devagar do que o equivalente nativo, a depender do navegador utilizado (Jangda *et al.*, 2019). Esse é exatamente o *gap* que este trabalho busca caracterizar no contexto específico de aplicações Rust, contribuindo com dados concretos para orientar decisões sobre quando a portabilidade do Wasm justifica a perda de desempenho em relação à execução nativa.

3 MATERIAL E MÉTODOS

A abordagem deste trabalho é quantitativa, buscando mensurar o desempenho e o consumo de recursos de diferentes algoritmos. Para isso, foram adotadas as etapas de pesquisa bibliográfica, implementação prática e análise comparativa.

3.1 PESQUISA BIBLIOGRÁFICA

Foram reunidas informações sobre Rust e WebAssembly na pesquisa bibliográfica, destacando seu contexto histórico, características técnicas e aplicabilidade.

Em relação ao Rust, foram abordados tópicos como o sistema de propriedade (*ownership*) e o modelo de empréstimo de valores (*borrowing*). Já para o WebAssembly, destacaram-se aspectos como o gerenciamento de memória, a segurança proporcionada pela restrição de permissões e sua integração com Rust por meio da portabilidade de código utilizando ferramentas como o *wasm-pack*.

3.2 IMPLEMENTAÇÃO PRÁTICA

Na etapa de implementação, foram utilizados cinco algoritmos distintos em Rust, cada um explora diferentes características de execução, como operações de inteiros e de pontos flutuantes, manipulação de *arrays*, laços de repetição, manipulação de *bits*, geração de número pseudo aleatórios e saída de dados. Os algoritmos foram obtidos do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026) e sofreram adaptações mínimas para serem executadas no ambiente de teste. Buscou-se utilizar o mínimo possível de bibliotecas externas, limitando-se à biblioteca Rayon para processamento paralelo e às bibliotecas nativas da linguagem, a fim de reduzir interferências externas nos resultados de desempenho.

No contexto do Rust, foi desenvolvido um código de entrada responsável por receber, por meio de parâmetros posicionais, o algoritmo a ser executado e a quantidade de execuções. Esse código também realiza a coleta do tempo de início e término da execução, mantendo os algoritmos isolados do processo de metrificação.

Para o WebAssembly, os algoritmos implementados em Rust foram portados utilizando a ferramenta *wasm-pack*. Além disso, foi criado um arquivo de entrada em HTML para carregar uma página contendo código JavaScript capaz de executar os algoritmos portados. Essa página também aceita parâmetros via URL para definir o algoritmo e a quantidade de execuções. A execução ocorre em um navegador sem interface gráfica (*headless*), utilizando a biblioteca Puppeteer de JavaScript.

3.2.1 Algoritmos

Os seguintes algoritmos foram utilizados na implementação prática:

- *fannkuch-redux*: foca em operações com inteiros e manipulação de *arrays*;
- *n-body*: realiza operações de ponto flutuante;
- *spectral-norm*: concentra-se em laços de repetição;
- *mandelbrot*: faz a manipulação de *bits*;
- *fasta*: avalia a capacidade de geração de números pseudo aleatórios e saída de dados.

3.2.2 Configuração dos Testes

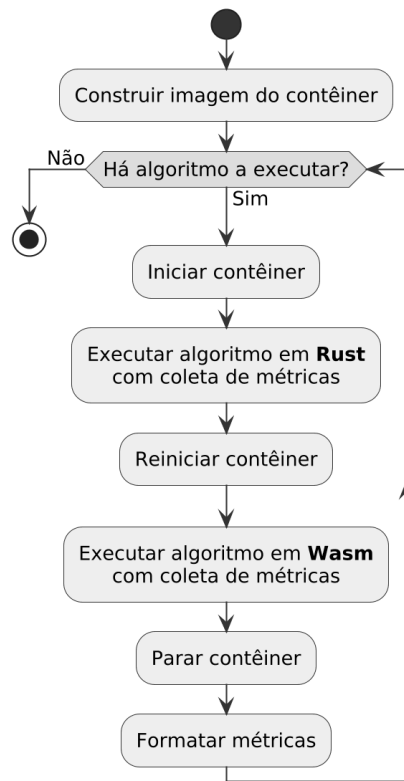
Cada algoritmo foi executado 10 vezes em sequência. Antes da coleta de métricas de cada algoritmo, ele foi executado 1 vez, com o objetivo de diminuir a influência do JIT e do *cache*. Os algoritmos foram executados com uma única *thread* nos dois ambientes, limitando a biblioteca Rayon por meio da variável de ambiente `RAYON_NUM_THREADS`, pois o processamento paralelo no WebAssembly exige a biblioteca `wasm-bindgen-rayon`, que não pôde ser compilada no ambiente de testes. Dessa forma, os dois ambientes executaram a mesma carga, com o mesmo número de *threads*. A parametrização dos algoritmos foi a mesma usada nos testes de desempenho do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026). O fluxo de execução dos testes e coleta de métricas dos algoritmos é ilustrado na Figura 1.

3.3 AMBIENTE DE TESTE

Com o objetivo de isolar o ambiente de execução, foram utilizados contêineres por meio do Docker, com quantidades controladas de CPUs e memória RAM, limitadas a dois núcleos de processamento e 2 GB de memória, visando aumentar a reprodutibilidade dos experimentos.

Os testes foram realizados em 3 computadores diferentes, sendo posteriormente calculada a média dos resultados obtidos. Além disso, foram padronizadas as versões das tecnologias utilizadas em todos os computadores. A seguir, são apresentados os hardwares e os softwares utilizados.

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos



Fonte: Os autores

3.3.1 Hardwares

Computador 1:

- Sistema operacional: Ubuntu 24.04 (Noble);
- Kernel: Linux 6.17.0-35-generic;
- Processador: Intel i7 1165G7, 4 núcleos / 8 threads;
- Memória RAM: 16 GB LPDDR4 (8x2 GB) 4267 MHz;
- Armazenamento: SSD NVMe de 1 TB Micron MTFDHBA1T0QFD.

Computador 2:

- Sistema operacional: Ubuntu 24.04 LTS (Noble Numbat) via WSL 2;
- Kernel: Linux 6.6.87.2-microsoft-standard-WSL2;
- Processador: Intel Core i5-10300H, 4 núcleos / 8 threads;

- Memória RAM: 8 GB DDR (8x1 GB) 3200 MHz;
- Armazenamento: SSD SATA de 500 GB KINGSTON SA400S37.

Computador 3:

- Sistema operacional: Ubuntu 22.04 (Jammy) via WSL;
- Kernel: Linux 6.6.114.1-microsoft-standard-WSL2;
- Processador: AMD Ryzen 5 5600x, 6 núcleos / 12 threads;
- Memória RAM: 32 GB DDR4 (2x16 GB) 3200 MHz;
- Armazenamento: SSD NVMe de 500 GB KINGSTON SNVS500G.

3.3.2 Softwares

- Docker: versão 29.5.3;
- Docker Compose: versão v5.1.4;
- Rust: versão 1.95.0;
- wasm-pack: 0.14.0;
- wasm-bindgen: 0.2.121;
- Rayon: 1.8;
- Puppeteer: 24.43.1;
- Chrome for Testing (modo *headless*): 148.0.7778.97;
- Node.js: v24;
- npm: 10.9.4.

3.4 BENCHMARKING

Inicialmente, foi considerada a utilização de bibliotecas como Criterion e Hyperfine para a realização do *benchmarking*. Entretanto, para simplificar a implementação, foram desenvolvidos *scripts* responsáveis por automatizar a coleta de métricas dentro do contêiner Docker, além de código em Python para formatação e processamento dos dados coletados.

Essa abordagem também permitiu alterar parâmetros dos testes por meio de arquivos de configuração e realizar ajustes finos, buscando reduzir ao máximo interferências externas nos resultados.

3.4.1 Coleta de métricas

Durante a avaliação, foram coletadas as métricas de tempo de execução, uso de memória ao longo do tempo e uso de CPU ao longo do tempo. Para reduzir a interferência do consumo de recursos em estado de espera (*standby*) pelo contêiner e, no caso do Wasm, pelo Puppeteer, além da sobrecarga de carregamento da página *web*, foi adotado um tempo de 10 segundos em estado ocioso (*idle*) antes e depois de cada execução. Além disso, o contêiner era reiniciado antes de cada execução para remover recursos residuais.

O tempo de execução foi medido pelo próprio código de entrada de cada ambiente, utilizando as funções de medição de tempo de cada linguagem: a estrutura *Instant* da biblioteca padrão do Rust, no ambiente nativo, e a função *performance.now()* do JavaScript, no ambiente WebAssembly. Essas funções medem intervalos de tempo sem sofrer influência de ajustes no relógio do sistema. Após a execução de aquecimento, foi registrado o tempo total das $N = 10$ execuções consecutivas, e o tempo médio de execução de cada algoritmo foi obtido pela Equação (3.1):

$$t = \frac{T_{\text{fim}} - T_{\text{início}}}{N} \quad (3.1)$$

em que $T_{\text{início}}$ e T_{fim} representam os instantes, em milissegundos, do início da primeira e do término da última execução medida.

O uso de memória e de CPU foi coletado por um *script* executado dentro do próprio contêiner, que realizava leituras periódicas dos contadores do *cgroup* do Linux, mecanismo do núcleo do sistema operacional responsável por contabilizar os recursos utilizados pelo contêiner. Entre as leituras, foi utilizado um intervalo de espera de 250 milissegundos, o que, somado ao tempo de leitura das métricas, resultou em uma taxa mediana entre 2,9 e 3,2 amostras por segundo nos três computadores, calculada pela razão entre o número de amostras e a duração de cada coleta. O *script* também registrava o consumo individual de cada processo, porém, na análise, foram consideradas apenas as amostras referentes ao contêiner como um todo.

A utilização de CPU na i -ésima amostra foi calculada pela Equação (3.2):

$$cpu_i = \frac{u_i - u_{i-1}}{\tau_i - \tau_{i-1}} \times 100 \quad (3.2)$$

em que u_i representa o tempo acumulado de CPU consumido pelo contêiner, em microssegundos, e τ_i o instante da leitura, também em microssegundos. Como o contêiner dispunha de dois núcleos, o valor pode ultrapassar 100%.

Embora coletada, a utilização de CPU não foi analisada como resultado comparativo entre os ambientes, sendo empregada apenas para separar os períodos

de repouso e de execução na análise de memória, descrita na Seção 3.4.3. Durante a execução, a utilização média permaneceu praticamente constante entre os algoritmos em cada computador e ambiente, como no computador 1, em que variou de 107,0% a 107,9% no Rust e de 123,0% a 123,9% no Wasm, de modo que a métrica não diferenciou os algoritmos avaliados. Além disso, nas amostras do período ocioso anterior à execução, cerca de 70% da utilização registrada para o contêiner não correspondeu a nenhum dos processos registrados individualmente, proporção obtida pela diferença entre a utilização do contêiner e a soma das utilizações dos processos, possivelmente em razão dos processos auxiliares de curta duração criados pelo próprio *script* de coleta a cada leitura, o que impede separar com segurança o consumo dos ambientes avaliados da interferência do processo de medição.

O consumo de memória na i -ésima amostra foi calculado pela Equação (3.3):

$$m_i = \frac{M_i - C_i}{2^{20}} \quad (3.3)$$

em que M_i representa a memória total contabilizada para o contêiner, em *bytes*, e C_i a parcela de *cache* de arquivos inativa, que pode ser liberada pelo sistema operacional a qualquer momento e, portanto, não representa consumo efetivo. A divisão por 2^{20} converte o valor para MiB.

3.4.2 Tempo de execução

Para a análise de desempenho temporal, o *overhead* percentual do WebAssembly em relação ao Rust nativo foi calculado pela Equação (3.4), e o fator multiplicativo, que indica quantas vezes a execução em WebAssembly foi mais lenta que a nativa, pela Equação (3.5):

$$\text{overhead} (\%) = \left(\frac{t_{\text{wasm}}}{t_{\text{rust}}} - 1 \right) \times 100 \quad (3.4)$$

$$\text{fator} = \frac{t_{\text{wasm}}}{t_{\text{rust}}} \quad (3.5)$$

em que t_{rust} e t_{wasm} representam os tempos médios de execução, em milissegundos, de cada ambiente.

3.4.3 Consumo de memória

Para a análise de memória, foram derivadas três métricas a partir das amostras coletadas em cada execução: *baseline*, pico e consumo líquido. Para isso, foi necessário separar o período ocioso do período de execução ativa. Como os algoritmos foram executados com uma única *thread*, a execução ativa

ocupa integralmente um núcleo de processamento, o que corresponde a 100% na Equação (3.2), elevando a utilização de CPU do contêiner acima desse valor. Não foi adotado um limiar inferior para identificar a execução porque, mesmo sem nenhum algoritmo em execução, a utilização de CPU do contêiner no período ocioso anterior à execução foi em média de 20,7% no Rust e 46,6% no Wasm, chegando a 91,3%.

Para evitar que picos momentâneos de CPU, como os causados pela inicialização do navegador, fossem confundidos com a execução do algoritmo, o período de execução foi definido como o intervalo entre o primeiro e o último trecho de $k = 8$ amostras consecutivas com utilização de CPU acima de 100%, número de amostras que corresponde a cerca de 2 a 3 segundos de coleta. Sendo a o índice da primeira amostra desse intervalo e b o da última, o conjunto de execução E , que reúne as amostras desse intervalo com utilização de CPU acima de 100%, é dado pela Equação (3.6):

$$E = \{ i \mid a \leq i \leq b \wedge cpu_i > 100\% \} \quad (3.6)$$

O conjunto ocioso O reúne as amostras do período de repouso utilizadas no cálculo do *baseline*: as $n = 10$ últimas amostras anteriores ao início da execução (a) com utilização de CPU igual ou inferior a 100%, que correspondem ao final dos 10 segundos de repouso que antecedem cada execução. O conjunto é dado pela Equação (3.7), considerando-se apenas os n maiores índices que satisfazem a condição:

$$O = \{ i \mid i < a \wedge cpu_i \leq 100\% \} \quad (3.7)$$

O consumo de *baseline* (Equação (3.8)) corresponde ao mínimo entre as amostras do conjunto ocioso O , representando o consumo do contêiner em repouso. O consumo de pico (Equação (3.9)) corresponde ao máximo entre as amostras do conjunto de execução E , representando o consumo durante a execução ativa. O consumo líquido é dado pela diferença entre pico e *baseline* (Equação (3.10)):

$$baseline = \min\{ m_i \mid i \in O \} \quad (3.8)$$

$$pico = \max\{ m_i \mid i \in E \} \quad (3.9)$$

$$líquido = pico - baseline \quad (3.10)$$

Essa distinção é necessária porque o contêiner Wasm mantém o Chrome Headless e o Puppeteer em execução permanente, elevando o *baseline* a cerca de 210 MiB independentemente do algoritmo executado.

Para verificar se os resultados dependiam da escolha de k e n , as métricas de memória foram recalculadas para todos os pares formados por um valor de k entre 7 e 20 (14 valores) e um valor de n entre 3 e 11 (9 valores), totalizando 126 pares de parâmetros. Em todos eles, o consumo líquido médio variou no máximo 1,6 MiB em relação aos valores adotados ($k = 8$ e $n = 10$). Valores de k menores que 7 ou de n maiores que 11 passam a incluir o período de carregamento do navegador, no qual a memória ainda está em crescimento, e por isso não foram considerados.

3.4.4 Consolidação dos resultados

Os procedimentos anteriores foram aplicados separadamente aos dados de cada computador. Em seguida, para cada algoritmo e ambiente, os resultados dos $K = 3$ computadores foram consolidados pela média aritmética, conforme a Equação (3.11):

$$\bar{x} = \frac{1}{K} \sum_{j=1}^K x_j \quad (3.11)$$

em que x_j representa o valor obtido no j -ésimo computador para o tempo médio de execução (Equação (3.1)) ou para as métricas de *baseline*, pico e consumo líquido (Equações (3.8) a (3.10)). O *overhead* e o fator multiplicativo de cada algoritmo foram calculados a partir dos tempos médios consolidados, aplicando-se as Equações (3.4) e (3.5) a $\bar{t}_{\text{rust}}$ e $\bar{t}_{\text{wasm}}$. Para analisar a variação entre os computadores, o *overhead* também foi calculado separadamente para cada um, aplicando-se a Equação (3.4) aos seus próprios tempos médios. Os resultados de cada computador são apresentados no Apêndice A.

Por fim, para sintetizar o comportamento geral, foram calculadas a média e a mediana do *overhead* entre os $A = 5$ algoritmos avaliados. A média foi obtida pela Equação (3.12), e a mediana corresponde ao valor central do conjunto de *overheads* ordenado de forma crescente, conforme a Equação (3.13):

$$\bar{o} = \frac{1}{A} \sum_{k=1}^A o_k \quad (3.12)$$

$$\tilde{o} = o_{(\frac{A+1}{2})} \quad (3.13)$$

em que o_k representa o *overhead* do k -ésimo algoritmo e $o_{(j)}$ o j -ésimo menor valor entre eles. Como A é ímpar, a mediana corresponde ao terceiro menor *overhead*. A mediana foi incluída por ser menos sensível a valores extremos do que a média. Os resultados de tempo, *overhead* e memória foram apresentados com uma casa decimal, e o fator multiplicativo com três, de modo a preservar a correspondência entre as duas métricas, dada pela Equação (3.14):

$$\text{fator} = 1 + \frac{\text{overhead} (\%)}{100} \quad (3.14)$$

O processamento dos dados foi realizado por meio de *scripts* em Python desenvolvidos pelos autores.

4 ANÁLISE DOS RESULTADOS

As métricas coletadas foram analisadas com base no levantamento teórico realizado, buscando avaliar os aspectos de desempenho de maneira objetiva e fundamentada em cada cenário analisado. Os valores apresentados neste capítulo correspondem à média dos resultados obtidos nos três computadores descritos na metodologia.

Para o tempo de execução, foi comparado o resultado entre os dois ambientes, expressando a diferença como *overhead* percentual e fator multiplicativo.

Para o consumo de memória, foram aplicadas as métricas de *baseline*, pico e consumo líquido descritas anteriormente, a fim de isolar o consumo efetivo do algoritmo do *overhead* fixo do ambiente de execução Wasm.

A utilização de CPU, embora coletada, não é apresentada como resultado, pois foi empregada apenas para identificar os períodos de execução, pelos motivos descritos na Seção 3.4.3.

4.1 TEMPO DE EXECUÇÃO

A Tabela 1 apresenta os tempos médios de execução obtidos nos ambientes Rust e WebAssembly para os cinco algoritmos avaliados, além do *overhead* percentual e do fator multiplicativo, calculados pelas Equações (3.4) e (3.5) a partir das médias dos três computadores (Equação (3.11)). Os resultados mostram que o *overhead* variou de 10,9% a 126,9%, indicando diferenças distintas entre os algoritmos avaliados. Em todos os casos, o WebAssembly apresentou tempo de execução maior que o Rust nativo.

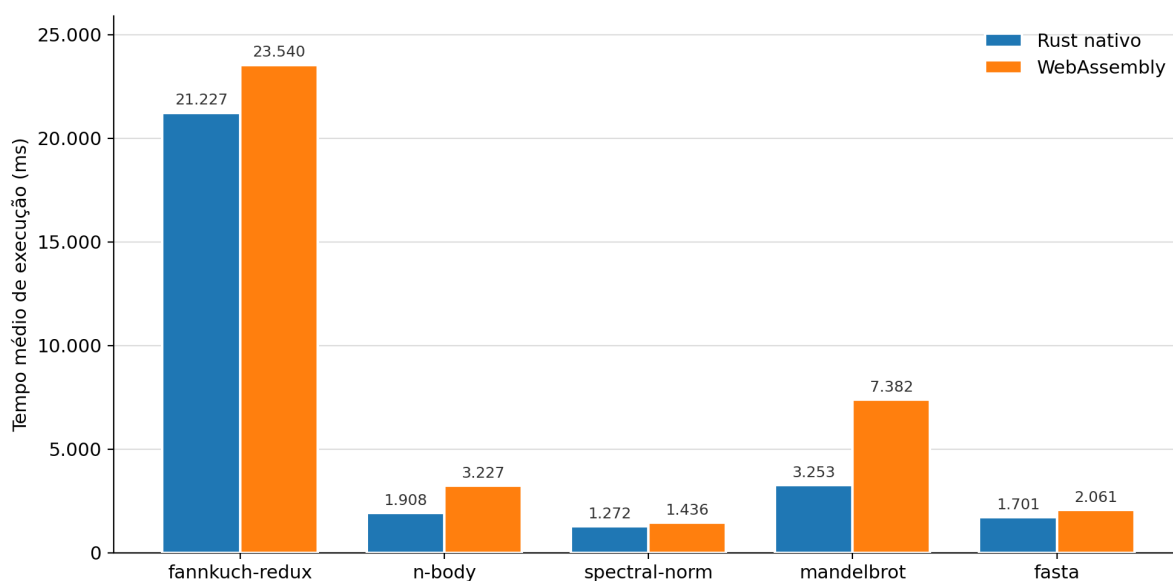
Tabela 1 – Comparativo de tempo de execução médio (ms)

Algoritmo	Rust (ms)	Wasm (ms)	Overhead (%)	Fator
fannkuch-redux	21.226,7	23.539,7	10,9	1,109x
spectral-norm	1.271,7	1.436,2	12,9	1,129x
fasta	1.701,5	2.061,5	21,2	1,212x
n-body	1.907,7	3.227,3	69,2	1,692x
mandelbrot	3.253,5	7.381,7	126,9	2,269x

Fonte: Os autores

Os menores *overheads* foram observados em fannkuch-redux e spectral-norm, com 10,9% e 12,9%, respectivamente. Em contraste, n-body e mandelbrot apresentaram as maiores diferenças, com 69,2% e 126,9%. A Figura 2 permite visualizar essas diferenças entre os tempos médios de execução nos dois ambientes.

Figura 2 – Comparativo de tempo de execução médio



Fonte: Os autores

O algoritmo fannkuch-redux registrou o menor *overhead*, correspondente a 10,9%, ou fator 1,109x. Trata-se de um algoritmo baseado em permutações de vetores de inteiros de tamanho fixo, com operações predominantemente inteiras.

O spectral-norm apresentou *overhead* de 12,9%. O algoritmo realiza operações de ponto flutuante de dupla precisão em laços sobre vetores.

O algoritmo fasta obteve *overhead* de 21,2%. Sua execução envolve a geração de números pseudoaleatórios, consultas a tabelas de probabilidade acumulada e a escrita das sequências geradas em blocos de memória.

O n-body registrou *overhead* de 69,2%. A simulação envolve cálculos iterativos de forças gravitacionais entre cinco corpos, com uso intensivo de operações de ponto flutuante de dupla precisão, incluindo raízes quadradas, sobre um conjunto reduzido de dados.

O mandelbrot apresentou o maior *overhead* observado, correspondente a 126,9%. Nesse cenário, o tempo de execução em WebAssembly foi mais que o dobro do observado na execução nativa. O algoritmo realiza processamento intensivo de ponto flutuante de dupla precisão e grava o resultado em um *buffer* de *bits*.

Considerando todos os algoritmos avaliados, o *overhead* médio foi de 48,2% (Equação (3.12)), enquanto a mediana foi de 21,2% (Equação (3.13)). A diferença entre essas métricas indica que os casos extremos, especialmente o mandelbrot, exercem influência significativa sobre a média geral dos resultados.

Os valores absolutos de tempo variaram entre os três computadores, que possuem configurações de *hardware* e sistema operacional distintas, e o *overhead* de alguns algoritmos também oscilou entre as máquinas, como no fannkuch-redux, entre

4,0% e 18,0%, e no spectral-norm, entre 7,6% e 26,6%. Ainda assim, a separação entre os grupos se manteve em todas elas: mandelbrot e n-body registraram os maiores *overheads* nos três computadores, enquanto fannkuch-redux, spectral-norm e fasta permaneceram abaixo de 27%.

4.2 USO DE MEMÓRIA

As métricas de *baseline*, pico e consumo líquido foram calculadas pelas Equações (3.8) a (3.10) para cada computador e consolidadas pela média entre eles (Equação (3.11)). As Tabelas 2 e 3 apresentam os resultados obtidos.

Tabela 2 – Consumo de memória do executável Rust (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	16,6	17,7	1,1
n-body	7,9	8,6	0,7
spectral-norm	8,1	9,0	1,0
mandelbrot	8,0	40,4	32,3
fasta	7,9	9,1	1,2

Fonte: Os autores

Tabela 3 – Consumo de memória do executável WebAssembly (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	218,6	237,0	18,4
n-body	209,9	213,7	3,8
spectral-norm	209,6	211,7	2,2
mandelbrot	209,9	245,9	36,1
fasta	209,9	212,7	2,8

Fonte: Os autores

O *baseline* do ambiente Rust situou-se entre 7,9 MiB e 16,6 MiB, refletindo principalmente o consumo do contêiner em estado ocioso. O valor mais alto, observado no fannkuch-redux, decorre de um dos computadores, que registrou 32,1 MiB nessa execução, enquanto os demais ficaram próximos de 9 MiB. No ambiente WebAssembly, os valores de *baseline* variaram entre 209,6 MiB e 218,6 MiB, em razão da execução permanente do Chrome Headless e do Puppeteer.

Dessa forma, a comparação direta entre os valores totais de memória não é adequada para avaliar exclusivamente o impacto dos algoritmos. Por esse motivo, a análise foi concentrada no consumo líquido de memória.

Considerando essa métrica, o Rust apresentou consumo líquido entre 0,7 MiB e 1,2 MiB em quatro dos cinco algoritmos: fannkuch-redux, n-body, spectral-norm e fasta. A exceção foi o mandelbrot, com 32,3 MiB, valor praticamente idêntico nos

três computadores e compatível com o *buffer* de saída alocado pelo algoritmo, que armazena um *bit* por *pixel* de uma imagem de 16.000 por 16.000 *pixels*, totalizando cerca de 30,5 MiB.

No WebAssembly, o consumo líquido ficou entre 2,2 MiB e 3,8 MiB para n-body, spectral-norm e fasta, ligeiramente acima dos valores do Rust nativo. O mandelbrot também apresentou o maior consumo nesse ambiente, com 36,1 MiB, valor igualmente compatível com o *buffer* de saída, que no WebAssembly é alocado na memória linear do módulo. O fannkuch-redux registrou 18,4 MiB, porém com grande variação entre os computadores, de 4,2 MiB a 39,5 MiB.

De modo geral, o consumo líquido do WebAssembly foi maior que o do Rust nativo em todos os algoritmos.

4.3 SÍNTESE COMPARATIVA

Os resultados mostram que o Rust nativo foi mais rápido em todos os algoritmos e que o *overhead* do WebAssembly variou de forma expressiva conforme o algoritmo. Fannkuch-redux, spectral-norm e fasta ficaram entre 10,9% e 21,2%, enquanto n-body e mandelbrot chegaram a 69,2% e 126,9%, separação que se manteve nos três computadores.

Também o ambiente WebAssembly exige, em repouso, cerca de 200 MiB a mais que o ambiente nativo, com *baseline* de cerca de 210 MiB, contra 7,9 MiB a 16,6 MiB no Rust, para manter o Chrome Headless através do Puppeteer em execução, valor praticamente constante entre os algoritmos e independente da carga processada, tornando a comparação direta com o ambiente nativo inadequada. Por esse motivo, a análise de consumo de memória foi concentrada no consumo líquido, que isolou o impacto do algoritmo em execução. Por essa métrica, os dois ambientes apresentaram consumo reduzido na maioria dos algoritmos, com o WebAssembly ligeiramente acima do Rust nativo, e o maior consumo em ambos foi o do mandelbrot, determinado pelo tamanho do *buffer* de saída.

4.4 LIMITAÇÕES DO ESTUDO

Há limitações que devem ser consideradas neste estudo:

- Nos testes realizados, a variável de ambiente `RAYON_NUM_THREADS` foi fixada em 1, restringindo a execução de algoritmos que utilizam a biblioteca *Rayon* a uma única *thread*, mesmo naqueles casos em que o processamento paralelo poderia representar ganhos de desempenho. Essa restrição foi necessária em razão de erros encontrados na compilação da biblioteca *wasm-bindgen-rayon* para o

ambiente WebAssembly, e foi replicada no ambiente Rust nativo para manter a padronização entre as comparações.

- Estudos futuros poderiam empregar ambientes de execução WebAssembly independentes de navegador, como *Wasmer* e *Wasmtime*, permitindo isolar de forma mais precisa o desempenho do código WebAssembly.
- A avaliação do ambiente WebAssembly limitou-se ao navegador Google Chrome, controlado por meio do Puppeteer, o que restringe a generalização dos resultados obtidos. Diferentes navegadores utilizam motores de execução WebAssembly distintos, cada um com estratégias próprias de otimização, o que pode alterar significativamente os resultados de desempenho.

5 CONSIDERAÇÕES FINAIS

Este trabalho realizou uma análise comparativa entre a execução nativa de código Rust e sua execução compilada para WebAssembly, avaliando os impactos em tempo de execução e consumo de memória. A utilização de CPU também foi coletada, mas mostrou-se pouco informativa para a comparação, pelos motivos descritos na Seção 3.4.3, sendo utilizada apenas para identificar os períodos de execução.

Para embasar essa análise, foram abordados os fundamentos da linguagem Rust e as características do WebAssembly na revisão da literatura. Além disso, na etapa prática foram selecionados, do The Computer Language Benchmarks Game, 5 algoritmos de diferentes classes de problemas, implementados em Rust, portados para WebAssembly e executados em ambiente controlado via Docker em 3 máquinas distintas. Os resultados mostraram que o Rust nativo foi mais rápido que o WebAssembly em todos os algoritmos avaliados, com *overhead* do WebAssembly entre 10,9%, no *fannkuch-redux*, e 126,9%, no *mandelbrot*, e mediana de 21,2%. A análise de memória evidenciou que parte do consumo do ambiente WebAssembly não está associada à execução dos algoritmos, mas ao ambiente de execução, que mantém componentes como o Chrome Headless e o Puppeteer em funcionamento contínuo e eleva o consumo de memória em repouso para cerca de 210 MiB.

Diante disso, a utilização das métricas de *baseline* e consumo líquido mostrou-se fundamental para isolar o impacto real dos algoritmos, permitindo uma análise mais precisa e justa entre os ambientes. Por essa métrica, o consumo líquido dos dois ambientes foi reduzido na maioria dos algoritmos, com o WebAssembly ligeiramente acima do Rust nativo, e o maior consumo em ambos foi o do *mandelbrot*, determinado pelo tamanho do *buffer* de saída. Observou-se ainda que o *overhead* de tempo variou de forma expressiva entre os algoritmos, com separação consistente nos três computadores.

Conclui-se, portanto, que o WebAssembly pode representar uma alternativa viável à execução nativa para aplicações Rust executadas no navegador, com custo adicional entre 10,9% e 21,2% em três dos cinco algoritmos avaliados. Em outras cargas, porém, o custo chegou a superar o dobro do tempo nativo, de modo que, quando o desempenho é o principal requisito, a adoção do WebAssembly deve ser precedida da medição com a carga de trabalho específica da aplicação.

Como limitação, os algoritmos foram executados com uma única *thread* nos dois ambientes, pois não foi possível habilitar o processamento paralelo no WebAssembly. Como trabalhos futuros, sugere-se avaliar a execução multi-*thread* no navegador, ampliar o conjunto de algoritmos e comparar outros mecanismos de execução de WebAssembly.

REFERÊNCIAS

- Bugden, W.; Alahmar, A. Rust: The programming language for safety and performance. **arXiv**, v. 2206.05503, 2022. Disponível em: <<https://arxiv.org/abs/2206.05503>>. Citado 7 vezes nas páginas 12, 13, 15, 16, 17, 21 e 22.
- Fulgham, B.; Gouy, I. **The Computer Language Benchmarks Game**. 2026. Acesso em: 18 Jun. 2026. Disponível em: <<https://benchmarksgame-team.pages.debian.net/benchmarksgame/>>. Citado 3 vezes nas páginas 10, 24 e 25.
- Haas, A. *et al.* Bringing the web up to speed with webassembly. **Communications of the ACM**, v. 61, n. 1, p. 107–115, 2017. Disponível em: <<https://dl.acm.org/doi/abs/10.1145/3062341.3062363>>. Citado na página 18.
- Jangda, A. *et al.* Not so fast: Analyzing the performance of webassembly vs. native code. **Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)**, p. 107–120, 2019. Disponível em: <<https://www.usenix.org/conference/atc19/presentation/jangda>>. Citado 3 vezes nas páginas 11, 22 e 23.
- MDN Web Docs. **WebAssembly**. 2026. Acesso em: 18 jun. 2026. Disponível em: <<https://developer.mozilla.org/en-US/docs/WebAssembly>>. Citado na página 18.
- Moric, Z.; Branstett, L.; Petrunic, R. Rust and webassembly for fast, secure, and reliable software. **Proceedings of the 33rd DAAAM International Symposium on Intelligent Manufacturing and Automation**, DAAAM International, Vienna, Austria, v. 33, p. 165–171, 2022. Disponível em: <<https://doi.org/10.2507/33rd.daaam.proceedings.023>>. Citado 3 vezes nas páginas 18, 19 e 22.

APÊNDICE A – RESULTADOS POR COMPUTADOR

Este apêndice apresenta os resultados obtidos em cada um dos três computadores descritos na Seção 3.3, antes da consolidação pela média (Equação (3.11)). A Tabela A1 apresenta os tempos médios de execução (Equação (3.1)) e o *overhead* de cada computador (Equação (3.4)), e a Tabela A2 apresenta as métricas de memória (Equações (3.8) a (3.10)).

Tabela A1 – Tempo médio de execução e *overhead* por computador

Algoritmo	Computador	Rust (ms)	Wasm (ms)	Overhead (%)
fannkuch-redux	1	18.923,1	19.681,9	4,0
	2	29.883,5	35.249,0	18,0
	3	14.873,4	15.688,1	5,5
spectral-norm	1	1.006,6	1.115,9	10,9
	2	1.910,4	2.055,9	7,6
	3	898,1	1.136,9	26,6
fasta	1	1.438,8	1.716,1	19,3
	2	2.407,7	2.973,1	23,5
	3	1.257,9	1.495,2	18,9
n-body	1	1.525,0	2.432,3	59,5
	2	2.563,9	4.856,6	89,4
	3	1.634,3	2.393,0	46,4
mandelbrot	1	2.839,4	5.994,5	111,1
	2	4.940,5	11.589,0	134,6
	3	1.980,6	4.561,7	130,3

Fonte: Os autores

Tabela A2 – Consumo de memória por computador (MiB)

Algoritmo	Computador	Rust			Wasm		
		Base	Pico	Líquido	Base	Pico	Líquido
fannkuch-redux	1	8,1	9,2	1,1	223,5	227,7	4,2
	2	32,1	33,3	1,2	226,1	265,6	39,5
	3	9,5	10,4	0,9	206,2	217,8	11,6
spectral-norm	1	8,2	9,3	1,0	222,8	224,3	1,5
	2	8,0	8,7	0,8	209,8	213,0	3,2
	3	8,0	9,1	1,1	196,2	198,0	1,8
fasta	1	8,1	9,2	1,1	223,0	225,7	2,7
	2	7,9	8,6	0,8	210,1	213,6	3,5
	3	7,7	9,3	1,7	196,6	198,7	2,1
n-body	1	8,4	8,7	0,3	222,7	225,4	2,6
	2	7,2	8,3	1,1	209,9	216,1	6,2
	3	8,0	8,7	0,7	197,1	199,8	2,7
mandelbrot	1	8,2	40,7	32,4	222,2	257,2	35,1
	2	7,8	40,1	32,3	209,9	246,8	36,9
	3	8,1	40,3	32,3	197,5	233,8	36,3

Fonte: Os autores